National University of Computing and Emerging Sciences - CFD Campus

Parallel and Distributed

Computing

Spring-2026

Assignment # 02

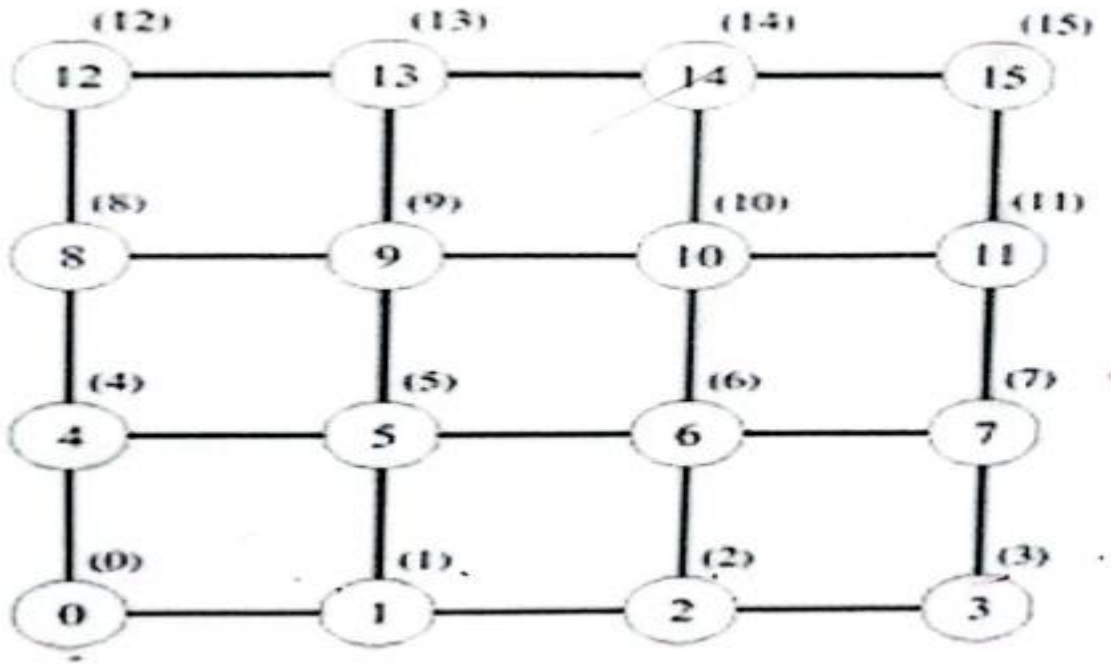
Submission Guidelines:

1. Submit your assignment as hardcopy (Handwritten) in class as well as in soft copy on Google Classroom. Please submit your file in this format **23F_XXXX_A2.**
2. The assignment's Section A should be on A4 pages.
3. Deadline for submission is (7-4-2026) 12:00 PM.
4. Do not submit your assignment after the deadline. Late submission will not be accepted.
5. Write concise answers; unnecessary explanations may lead to negative marking.
6. In case of plagiarism zero marks will be awarded & Also read carefully.

Section (A)

Question # 01. (15)

Perform circular 6-shift on the following 16 node-Mesh, perform calculation on right side and Illustrate arrows for direction of shift.



Question # 02. (15)

Speculative Parallel Execution and Rollback Synchronization

Modern high-performance systems often predict task outcomes to increase parallelism through speculative execution, where some threads execute branches before dependencies are confirmed.

(a) Explain how speculative decomposition differs from recursive or data decomposition. How does speculation expose additional concurrency beyond the original dependency graph?

(b) When a speculation fails (e.g., predicted dependency was wrong), threads must roll back to a consistent global state. Discuss how checkpoint-based rollback synchronization ensures correctness while minimizing wasted computation.

(c) Propose a hybrid scheduling policy that balances speculation depth and rollback cost to achieve maximum throughput under unpredictable task dependencies.

Question # 03. **(15)**

Hierarchical Task Mapping on Heterogeneous Multi-Level Systems

Large-scale platforms combine CPUs, GPUs, and distributed memory, requiring multi-level mapping strategies for efficient parallel execution.

(a) Describe how a task-dependency graph can be partitioned hierarchically into clusters for CPUs (coarse granularity) and threads within each cluster (fine granularity). Why is single-level mapping insufficient here?

(b) When communication cost between clusters is significantly higher than intra-cluster computation, derive the condition under which hierarchical load balancing minimizes total execution time.

(c) Suggest a dynamic mapping algorithm that uses real-time performance feedback to reassign subtasks across heterogeneous nodes without violating dependency or data-locality constraints.

Question # 04 **(15)** [**Coding + GUI + Deployment]**

Mesh Circular Shift Visualizer — Interactive Web Application

Background

In parallel computing, a circular q-shift is a fundamental permutation operation where node i transfers its data to node (i + q) mod p. On a 2D mesh topology, this is implemented efficiently in two stages:

- **each node shifts within its row by (q mod √p) positions**Stage 1 — Row Shift:
- **each node shifts within its column by ⌊ q / √p⌋ positions**Stage 2 — Column Shift:

You are required to build a full-stack interactive web application that simulates and visualizes this operation, deploy it publicly, and submit a technical report.

Deliverables (What to Submit)

D1. GitHub Repository — public repo with all source code, README, and commit history
D2. Live Deployment URL — application deployed publicly, accessible via browser
D3. A live public URL is MANDATORY. No live link = 0 marks for the entire question.
D4. Technical PDF Report — submitted on Google Classroom alongside code link

Part (a) — Application Development [6 marks]

Build a web application (any stack of your choice) that implements and visualizes circular shift on a 4×4 mesh. Your app must include:

(i) Input Controls: A UI panel where the user can enter values for p (4–64, perfect square only) and q (1 to p−1). Include validation.
(ii) Mesh Grid Visualization: Render the p nodes as a √p × √p grid. Each node displays its index and current data value.
(iii) Step-by-Step Animation: Show Stage 1 (row shift) and Stage 2 (column shift) as animated steps with arrows indicating data movement direction.
(iv) Before/After State: Clearly display the node data before the shift, after Stage 1, and after Stage 2 (final state).

Part (b) — Complexity Analysis Panel [3 marks]

Inside the application, add a real-time Complexity Panel that updates as the user changes p and q:

- Display: Row shift amount, Column shift amount, and Total communication steps
- Show the formula: Ring steps = $\min\{q, p-q\}$ vs Mesh steps = $(q \bmod \sqrt{p}) + \lfloor q/\sqrt{p} \rfloor$
- Render a small bar chart or numeric comparison showing Mesh vs Ring step count

The user should clearly see why the mesh is more efficient than a naive ring shift.

Part (c) — GitHub Repository Structure [2 marks]

Your GitHub repository must follow this structure:

```
mesh-shift-visualizer/
├── public/
├── src/
│   ├── components/
│   │   ├── MeshGrid.*       ← grid rendering + animation
│   │   ├── ControlPanel.*   ← user inputs
│   │   └── ComplexityPanel.*← analysis panel
│   ├── utils/
│   │   └── shiftLogic module      ← pure shift algorithm (testable)
│   ├── App.*
│   └── index.js
├── README.md  ← setup instructions + live deployment URL
└── package.json
```

Commit history must show meaningful incremental commits — not a single bulk commit.

Part (d) — Technical PDF Report [4 marks]

Submit a PDF report (3–5 pages) containing all of the following sections:

Report Section	Required Content	Pages
1. Introduction	Brief problem statement, what circular shift is, and why visualization helps	~0.5
2. Algorithm Description	Explain 2-stage mesh algorithm with formulas. Include a worked example for p=16, q=5 showing each node's data movement	~1
3. Application Architecture	Component diagram, data flow, state management approach	~0.5
4. Complexity Analysis	Derive and compare Mesh steps vs Ring steps. Table for p=16,64 with q=3,5,7	~1
5. Deployment Guide	Step-by-step: GitHub setup, deployment steps, and how to run locally	~0.5
6. Screenshots	At least 3 screenshots: initial state, mid-animation, final state. Include your live deployment URL visibly in one screenshot	~0.5

Report file naming format: 23F-XXXX_A2_Q4_Report.pdf

Marking Rubric

Criterion	Marks	Notes
Part (a) — Web app: grid, controls, animation	6	Must work on live public URL
Part (b) — Complexity panel with comparison	3	Formulas must be correct
Part (c) — GitHub repo structure + commit history	2	Must be public repo
Part (d) — PDF report completeness	4	All 6 sections required
PENALTY: No live public URL	−15	Entire question = 0 marks

PENALTY: No GitHub repo / private repo	−2	Mandatory deliverable
TOTAL	15	

Tech Stack: Any framework or language is allowed — React, Vue, Vanilla JS, Python Flask, Django, etc. Backend is optional.

Deployment: Push to GitHub → Deploy on Vercel/Netlify (frontend) or Render/Railway (Flask/backend). All free tiers sufficient.

Question # 05 **(15)**

Granularity, Critical Path, and Average Degree of Concurrency

Consider a parallel program decomposed into 8 tasks with the following execution times and dependencies:

Task	Exec Time	Depends On
T1	3	—
T2	4	—
T3	2	T1
T4	5	T1, T2
T5	3	T2
T6	4	T3, T4
T7	2	T4, T5
T8	3	T6, T7

(a) Draw the task-dependency graph for the above tasks, clearly labeling each node with its task ID and execution time. Identify all start nodes and finish nodes.

(b) Identify the critical path in this task-dependency graph. What is the critical path length?

(c) Calculate the total work (sum of all task execution times) and the average degree of concurrency. What does this metric tell you about the efficiency of this decomposition?

(d) If you could split T4 into two equal sub-tasks T4a and T4b (each of time 2.5) that can run in parallel, how would this affect the critical path length and average degree of concurrency? Is this always beneficial?

Question # 06 **(15)**

All-to-All Broadcast vs. One-to-All Broadcast: Architecture Trade-offs

Modern parallel systems implement collective communication operations differently based on the underlying network topology. This question explores the cost differences between one-to-all and all-to-all broadcast on three architectures.

(a) Compare the communication complexity (number of steps and messages) of one-to-all broadcast and all-to-all broadcast on a p-node ring. Use the recursive doubling approach for one-to-all and the circular shifting approach for all-to-all. Express costs as a function of p and message size m.

(b) For a $\sqrt{p} \times \sqrt{p}$ mesh, explain how all-to-all broadcast is performed in two phases (row-wise then column-wise). Derive the total number of messages each node sends and receives. Why does the mesh algorithm achieve better bandwidth utilization compared to the ring?

(c) In a practical cluster with 64 nodes, the network administrator must choose between ring,

mesh (8×8), and hypercube (6-dimensional) topologies for a scientific simulation that relies heavily on all-to-all broadcast of 1 KB messages. Using the communication cost formulas from your course, calculate the total data volume each node handles for each topology, and recommend the best topology with justification.

Section (B)

Question # 01

Topology-Aware All-to-All Personalized Communication Coding Part (Python)

In parallel computing, **All-to-All Personalized Communication** (Total Exchange) is one of the most dense communication patterns, where every node sends a unique, distinct message to every other node. In this task, you will implement and visualize this pattern across three distinct interconnection topologies: **Ring, 2D Mesh, and Hypercube.**

Given Constraints:

Node Count (p): Must be a power of 2 ($p=2^d$).

Initial State: Each node i contains (p−1) distinct messages, where msg[i][j] is intended only for node j.

Message Size (m): Default is 1024 bytes.

Goal: Move all msg[i][j] to their correct destinations using the most efficient route for that specific topology.

Part A: Topology Construction & Visualization [2 Marks]

Using Python libraries (such as **networkx** for graph logic and matplotlib or **Pygame** for animation), create a visual representation of the following:

Ring: A circular layout where each node i is connected to (i+1)(modp).

2D Mesh: A sqrt (p) × sqrt (p) grid with row and column links.

Hypercube: A d-dimensional graph where nodes are connected if their binary IDs differ by exactly one bit.

Visual Requirement: Use colored markers or "packets" to animate the movement of data along the edges during the exchange process.

Part B: Algorithmic Implementation [3.5 Marks]

1. **Ring Implementation (Shift Algorithm) [1 Mark]**

   Logic: Implement the $p-1$ step shift algorithm. In each step k, node i sends the message intended for node $(i+k)(\mathrm{mod}\,p)$ to its neighbor.

   Metric: Track the total number of communication steps ($t=p-1$).

2. **2D Mesh Implementation (Two-Phase Routing) [1.5 Marks]**

   Phase 1 (Row-wise): Nodes perform all-to-all personalized communication only within their row to gather all messages destined for their target columns.

   Phase 2 (Column-wise): Nodes perform a second all-to-all exchange within their columns to deliver messages to the final destination.

   Visual Requirement: The animation must clearly show the Row Sweep completing before the Column Sweep begins.

3. **Hypercube Implementation (Dimension-Ordered) [1 Mark]**

   Logic: Execute communication in $d=\log_2 p$ steps. In step j, nodes exchange data across the jth dimension.

   Constraint: In each step, the size of the combined message being exchanged doubles as nodes **"pack"** messages for the next dimension.

Part C: Performance & Comparative Analysis [1.5 Marks]

Your script must generate a summary table and a bar chart comparing the three topologies based on the following:

TOPOLOGY	STEPS COUNTED	COMPLEXITY LOGIC	WHY?
RING	P-1	LINEAR	SIMPLE BUT SLOW FOR MANY NODES.
MESH	2(SQRT(P)-1)	SQUARE ROOT	BETTER THAN A RING; SPLITS WORK INTO ROWS/COLUMNS.
HYPERCUBE	LOG2 P	LOGARITHMIC	FASTEST STEPS, BUT HIGH "TRAFFIC" ON LINKS.

Question # 02

Circular Shift on a 2D Mesh (Toroidal Grid) Coding Part (Python)

Many parallel machines and on-chip networks organize processing elements in a 2D mesh or toroidal grid, where data movement is restricted to neighbor links. Real applications such as stencil codes, image processing, and domain decomposition frequently require circular shifts of data across rows and columns. In this task, you will implement a neighbor-only circular shift on a 2D toroidal mesh, carefully orchestrating row and column movements so that no data is overwritten and the final layout corresponds to a global K-step circular shift in row-major order.

Input

- Integers **R** and **C**: number of rows and columns (e.g., $R = C = 4$).
- Total nodes $P = R \times C$, with node IDs $0 \ldots P - 1$mapped in row-major order.
- Each node $k$initially stores a **distinct data item** equal to its ID (or a tuple containing ID and coordinates).
- A **circular shift distance** Kin **row-major order** (e.g., move each data item K positions forward around the toroidal mesh).

Parallel Execution

- Represent each mesh location (r, c)as a **worker** (thread or lightweight process).
- Build neighbor relations for a **toroidal mesh**:
  - Left/right with wrap-around in each row.
  - Up/down with wrap-around in each column.
- Implement the circular shift as a sequence of **neighbor exchanges only**:
  - Decompose the global K-step row-major shift into:
    1. **Row shifts** (horizontal circular shifts in each row).
    2. **Column shifts** (vertical circular shifts) to "compensate" and match the final layout
       (similar to the three steps in the slide: row shift + two column phases).
  - Use **temporary buffers / double buffering** so that in one step:
    - Every node sends to its neighbor and receives from another neighbor
    - No data is overwritten before it is sent.
- Optimize for large K:
  - Use $K' = K mod P$to avoid redundant full cycles.
  - Reduce the number of actual neighbor-shift iterations when possible (e.g., exploit symmetry in rows/columns).

Synchronization

- Use a **barrier** after each of the main phases:
  - Row shift phase.
  - First column shift phase.
  - Second column shift / final alignment phase.
- If using shared 2D arrays, protect write operations with **mutexes** or perform updates into private "next- grid" buffers and then swap references at the barrier.
- Ensure that all nodes in a row/column complete their sends/receives for the current step before proceeding to the next step.

Performance Analysis

- Measure and print:
  - Execution time for the full shift for different values of **R, C, and K**.
- Create tables/plots:
  - **Time vs K** for fixed R, C.
  - **Time vs P (R×C)** for fixed K.
- Estimate:
  - **Number of neighbor communications** per data item (as a function of R, C, K).
  - Approximate **critical path length** in terms of the number of row and column steps.
- Relate the results to:
  - How **communication along rows vs columns** affects performance.
  - How toroidal connectivity reduces edge effects compared to a plain mesh.

Expected Output

- Print the grid:
  - Initial data layout (showing which ID is at each (r, c)).
  - Final layout after the circular shift (showing where each ID ended up).
- A correctness check:
  - Verify the final grid is a **permutation** of `0…P-1`.
  - Verify that each item has moved exactly K positions in row-major order.
  - Print `"Mesh circular shift: CORRECT"` or `"INCORRECT"`.
- Tables/graphs showing:
  - Runtime behavior vs K and vs grid size.
- A short analysis (5–8 lines) explaining:
  - How limiting communication to **nearest neighbors** and using **double buffering** helps avoid data loss and reduces contention.
  - How the number of steps scales with grid size and K.

Question # 03

All-to-All Personalized Communication on a d-Dimensional Hypercube Coding Part (Python)

Modern high-performance clusters often use logical hypercube topologies to reduce communication steps in collective operations. In such systems, each processing element must exchange distinct data with every other element without overloading any single link. In this task, you will simulate all-to-all personalized communication on a d-dimensional hypercube and analyze how messages are progressively routed along dimensions while keeping the critical communication path logarithmic in the number of nodes.

Requirements

Input

- Integer **d** (hypercube dimension), with total nodes $P = 2^d$.
- Each node $i \in [0, P-1]$initially stores **personalized messages** for every other node:

$$\text{msg}(i,j) = (i,j), j \neq i$$

- Optionally: seed for the random generator (for reproducible timing experiments).

Parallel Execution

- Represent each hypercube node as a **worker** (thread or lightweight process).
- Build the **neighbor table** using binary labels: nodes are neighbors if their IDs differ in exactly one bit.
- Implement the **d-step hypercube all-to-all algorithm**:
  - At step k(0-indexed), node $i$exchanges messages only with the neighbor whose ID differs in bit k.
  - After the exchange:
    - Each node **keeps** messages whose destination matches its own ID in the lowest $k+1$bits.
    - It **stores** the remaining messages in a "forward buffer" for the later steps.
- Use **double buffering** per node: one buffer for messages being sent in this step and one for messages being accumulated for future steps.
- After **d steps**, each node must hold exactly the set of messages $\{(s,t) \mid s \neq t\}$destined for itself.

Synchronization

- Use a **barrier** after each dimension step $k$so that:
  - All sends/receives at step $k$complete before step $k+1$begins.
- Protect shared data structures (e.g., per-node queues if stored globally) with **mutex locks**.
- Ensure no node reads another node's buffer while it is being modified (avoid race conditions).

Performance Analysis

- Count and print, after each step:
  - Total number of messages still **not at their final destination**.
- Measure **wall-clock time** for:
  - Full all-to-all execution for a given d.
  - Different dimensions $d = 3,4,5, \ldots$.
- Plot (e.g., using matplotlib):
  - Step number vs. messages in transit.
  - **Total execution time vs. dimension d (or vs. P = 2^d)**.
- Approximate:
  - **Critical path length** as the number of steps d.
  - Message complexity per node and globally and relate it to the timing results.

Expected Output

- For each node t: list or count of final messages (s, t)verifying correctness.
- A statement:
  - `"Hypercube all-to-all: CORRECT"` or `"INCORRECT"` after automatic validation.
- Visulaization using Matplot:
  - Messages-in-transit per step.
  - Execution time vs. number of nodes (or dimension).
- A short-written analysis (5–8 lines) explaining:
  - Why the number of in-flight messages decrease each step.
  - How the hypercube structure leads to logarithmic critical-path length and good scalability.